

Job Digest Agent

Final build plan: agent, private app, project site (22 sessions)

Topics covered

- Goals, decisions and architecture (agent, private app, public site)
- Sources: company boards, search APIs, remote feeds, your own email alerts
- Your final profile and how matching, scoring and filtering work
- Phase 1: the agent, Sessions 0 to 12 (setup to scheduled daily digest on Telegram and email)
- Phase 2: private single-user app for use from any device, Sessions 13 to 17
- Phase 3: public project site (SEO, structured data, domain, audit), Sessions 18 to 21
- Anti-slop rules, audit checklist, risks, items to verify, daily checklist

Format of this document

- Plain black and white. Each session has a heading, bullet points for the work, and a Check line that is the pass/fail rule for that session.
- A horizontal line separates every session. Page numbers appear at the bottom of each page.
- Six small diagrams (sources, architecture, flow, repository tree, score weights, timeline) are drawn as real figures, not text.
- A matching Markdown file (Job_Digest_Agent_FINAL_Plan.md) holds the same content for your repository.

1. Overview and goals

A personal agent that finds as many relevant fresher, internship and apprenticeship jobs as it can, from permitted sources only, and delivers one digest of apply links every morning at about 08:00 IST to Telegram and email. You apply manually. A private, password-protected web app lets you use it from any device, and a small public site presents the project properly. It is for you only: one profile, one login, no other users.

- Find as many jobs as possible that fit you, then rank them so you read the best first.
- Deliver to both Telegram and email, with the email going to a different mailbox from your alerts inbox.
- Never apply for you, never scrape sites that forbid it, never claim a score is a hiring prediction.
- Stay usable from any device through a private hosted app.

How to work each session

- A session is one focused 2 to 3 hour block. Build only that session, run its Check, commit, move on.
- If a session takes double the time, cut scope (fewer sources, fewer pages), never the tests.
- Keys and your real profile live in .env, GitHub secrets or the database, never in Git or prompts.
- Commit message style: 'day8: eligibility + match score'.
- Before using any source, API or host, re-check its current terms, limits and availability that day.

2. Decisions

- Personal use only. No accounts for other people, no sign-up.
- Stack: Python 3.12, httpx, Pydantic (strict), PyYAML, SQLAlchemy + psycopg, Jinja2, FastAPI, pytest, GitHub Actions. No LLM in v1; the top-15 re-rank is an optional later extra.
- Database: Supabase Postgres, using the pooler connection string. The direct connection is IPv6-only and GitHub runners often lack IPv6. Verify in Session 1.
- Profile: profile.yaml is gitignored. Until Session 13 the scheduled run reads it from a repository secret (PROFILE_YAML); from Session 13 it lives in the database. Only profile.example.yaml is committed.
- Delivery: Telegram and email are both supported; channels are set in the profile. Jobs count as notified once at least one channel succeeds.
- Alerts mailbox vs notification mailbox: the agent reads mailbox A (your LinkedIn/Naukri alerts) over IMAP with an app password and sends the digest to mailbox B. IMAP is preferred because Google OAuth apps in testing mode reportedly issue refresh tokens that expire in about 7 days. Verify.
- Sources: public job-board APIs, official search APIs and your own email alerts. No scraping of LinkedIn, Naukri or Indeed, no browser automation, no auto-applying.
- Website: the public project site is static, generated by the same Python and Jinja2 code, with near-zero JavaScript. The private app is separate, noindex and behind login.

3. Architecture

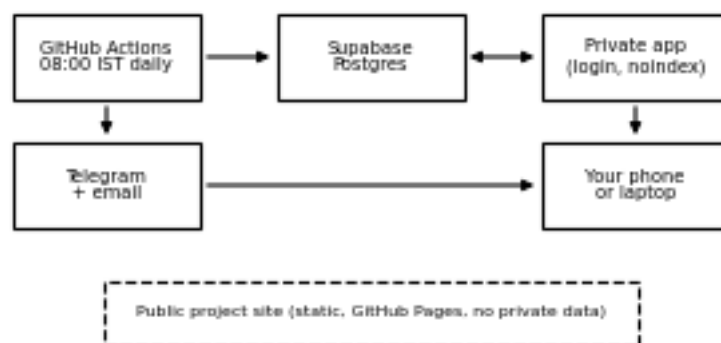


Figure 1. Where each part runs

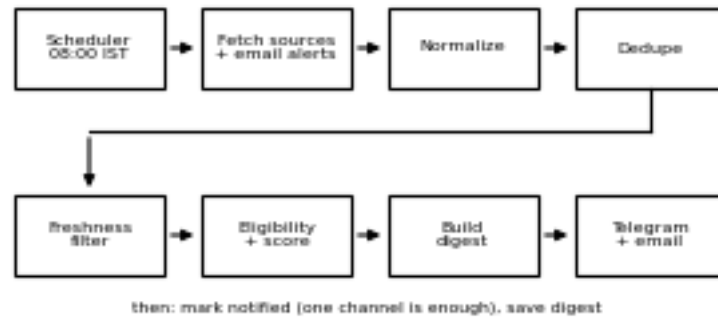


Figure 2. Daily run, from the 08:00 IST trigger to delivery

Scheduler (08:00 IST) -> fetch sources and email alerts -> normalize -> dedupe -> freshness filter -> eligibility + match score -> build digest (two sections) -> send Telegram and email -> mark notified once one channel succeeds -> save digest.

4. Sources

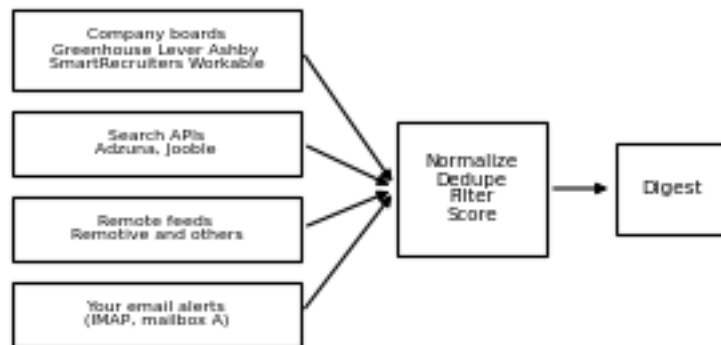


Figure 3. Sources feeding one pipeline

Source	How it is used	Notes
Greenhouse, Lever	Company boards: you list slugs, agent downloads every opening, filters locally.	Public JSON endpoints. Verify each slug.
Ashby	Company boards: public posting API per board name; skip unlisted postings.	No key. Slug from <code>jobs.ashbyhq.com/<name></code> .
SmartRecruiters	Company boards: public Postings API per company id.	Depends on the customer's plan; per-IP rate limits.
Workable	Company boards at <code>apply.workable.com/<subdomain></code> .	Verify the endpoint and terms on the day.
Adzuna	Keyword search, <code>country=in</code> , one query per role group.	Free key; check current limits.
Joooble (optional)	Aggregated search API, links route through Joooble.	Free key; check limits and terms.
Remotive and other remote feeds	Remote roles.	Remotive: max 4 fetches a day, link back and credit it in the digest.
Your email alerts	IMAP read of the alerts mailbox, parse links per sender.	Layouts change, so keep sample-email tests.
Manual add	Paste a link you found yourself.	Joins dedupe and tracking.

5. Repository layout

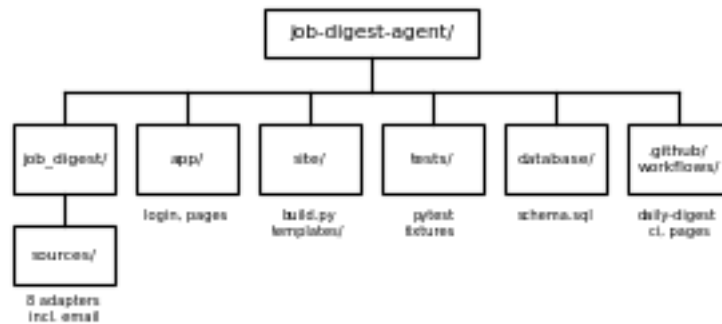


Figure 4. Repository tree

```

job-digest-agent/
  job_digest/  run.py models.py config.py db.py profile.py
               normalize.py dedup.py match.py digest.py notify.py
               sources/  greenhouse.py lever.py ashby.py smartrecruiters.py
                           workable.py adzuna.py remote.py email_alerts.py
  app/         main.py auth.py templates/ static/
  site/        build.py templates/ static/
  tests/       (fixtures: sample emails, API responses)
  database/schema.sql
  docs/decisions.md
  profile.example.yaml  sources.yaml  .env.example
  .github/workflows/  daily-digest.yml ci.yml pages.yml
  
```

Keep `sources/` inside `job_digest/` so the scheduler command `'python -m job_digest.run'` imports cleanly. `profile.yaml` is never committed.

6. Your profile and matching

Summary of the final `profile.yaml` (version 3). The real file stays private and is never committed.

Area	Setting
Education	B.Tech, computer science engineering, batch 2027.
Level	Fresher; up to 2 years of experience required. Internships and apprenticeships included.
Target roles (41 titles)	Software, web, frontend, full stack, backend, Python; ML, AI, NLP, LLM, generative AI, prompt engineer; data and business analyst; QA, tester, SDET; software, ML, AI, NLP, data science interns; apprentice and graduate apprentice.
Skills	HTML, CSS, JavaScript, C, C++, Python, Git, GitHub, React, machine learning, AI, data structures, algorithms, Supabase, FastAPI, NLP, LLM, SQL, PostgreSQL, REST API, Tailwind CSS, Linux, Azure, TypeScript, Docker, pandas, NumPy.
Tracked gaps (display only)	Java, Spring Boot, Angular, Node.js, scikit-learn, PyTorch. They fill the 'missing skills' line and never change the score.
Locations	Anywhere in India. Preferred: Noida, Delhi and NCR, Gurugram, Kolkata, Mumbai, Pune, Indore. Remote allowed.
Salary	Full-time: minimum 3.5 LPA when a salary is listed; if unlisted the job is kept and labelled. Internships: optional stipend minimum, currently off.
Hard filters	Title words (whole word, title only): senior, lead, manager, principal, director, staff, architect. Company blacklist. Required years above 2. A named batch year other than 2027.
Scoring	0 to 10: skills 4, role 3, level 2, location 1. Location: preferred city 1.0, elsewhere in India 0.5, remote 0.5, abroad 0. Minimum score 4; Strong is 7 and above.
Digest	Two sections: internships and apprenticeships (apply now), then full-time (after graduation). Up to 40 items. Telegram and email.

Small fixes to apply to the file

- Remove the bare alias 'rest' under rest api: it matches ordinary text such as 'the rest of the team'. Keep restful, restful api, rest apis, api integration.
- Leave the optional product manager roles commented out: 'manager' is a hard-exclude title word, so they would be dropped anyway.
- Drop the broad aliases 'containers' (Docker), 'azure devops' (a different tool) and, if false matches appear, 'ts' (TypeScript).
- Load the file with a strict Pydantic model (unknown keys rejected) so a typo fails loudly.

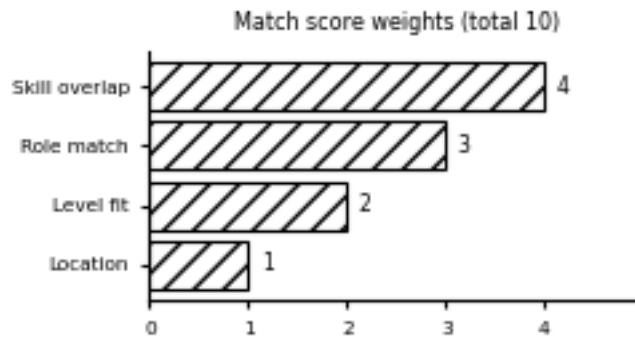


Figure 5. Score weights

The score is informational and is not a hiring prediction. Every job shows matched skills, missing skills, the location reason and the source. Strong matches are 7 and above; Maybe is 4 to 6. Salary or stipend is shown, or 'salary not listed'.

7. Timeline

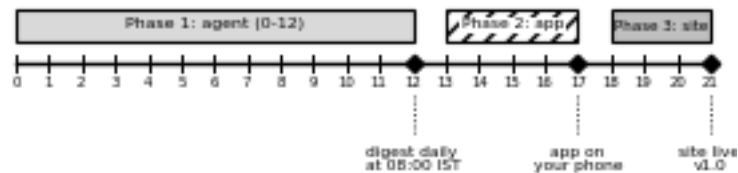


Figure 6. Sessions 0 to 21 with milestones

A session is one focused 2 to 3 hour block. With rest days, expect roughly 6 to 7 calendar weeks in total, with the daily digest working after about 3 to 4.

8. Phase 1: the agent (Sessions 0 to 12)

Session 0: Setup (~2h)

- Install Python 3.12, Git, VS Code. Create the GitHub repo, clone it, create and activate .venv.
- Add .gitignore (.venv, .env, profile.yaml, __pycache__), requirements.txt, pytest.
- Add ci.yml that runs pytest on push. Make the first commit, then stop for the day.

Check: A fresh clone + pip install -r requirements.txt + pytest is green.

Session 1: Profile + schema (~3h)

- Use the final profile.yaml (version 3). Commit only profile.example.yaml.
- Strict Pydantic model: unknown keys rejected, required fields enforced, lists deduplicated on load.
- database/schema.sql: job_sources, jobs, seen_jobs (fingerprint primary key), digests, source_runs, email_messages (processed Message-IDs).
- Connect through db.py using the pooler connection string. Store the profile as the PROFILE_YAML repo secret for now.

Check: A typo'd key and a missing required field both fail with a clear message; the tables exist.

Session 2: Source interface + Greenhouse and Lever (~3h)

- JobSource protocol: a name and an async fetch() returning a list of RawJob.
- sources.yaml with 20 to 30 verified company board slugs to start; adapters for the public Greenhouse and Lever JSON endpoints.
- Fetch sequentially with a small delay and a clear User-Agent. Filter location to India or remote after fetching.

Check: A script prints today's raw jobs; one bad slug logs a warning and the run continues.

Session 3: Ashby, SmartRecruiters, Workable (~3h)

- Ashby: public posting API per board name; skip postings that are not listed.
- SmartRecruiters: public Postings API per company id (availability depends on the customer's plan).
- Workable: boards at apply.workable.com/<subdomain>; verify the endpoint and terms before relying on it.
- Test each adapter against recorded responses. Grow the slug list toward 100+ over time.

Check: Each adapter passes its fixture test; a 404 slug is skipped with a warning.

Session 4: Adzuna, search and remote feeds (~3h)

- Adzuna with country=in: one query per role group (software and web, ML and AI, data and analyst, QA, intern and apprentice). Check the current free-tier limits first.
- Add one remote feed. For Remotive: at most 4 fetches a day, link back and credit it in the digest. Jooble is optional, after checking its terms.
- Wrap every source in try/except; record success, count and error in source_runs.

Check: Three or more sources return jobs; killing one source's network still yields results from the rest.

Session 5: Email alerts source (~3h)

- EmailAlertSource reads the alerts mailbox (A) over IMAP with an app password. Fetch only from the alert senders you list.
- Parse job links and titles per sender, using saved sample emails as test fixtures. Treat email content as untrusted.
- Record processed Message-IDs in the database. Never delete or move mail. The digest goes to mailbox B, never back into A.

Check: Parser tests pass on the saved samples; mail from unknown senders is ignored.

Session 6: Normalization (~3h)

- Job model: title, company, location, canonical_url, description_text, posted_at, source, skills[], min_years, is_intern, is_apprentice, salary_lpa or stipend, batch_years.
- Strip HTML, clean locations (Gurgaon, Calcutta, Delhi NCR), canonicalize URLs, parse dates.
- Skill extraction with word-boundary regex from your profile skills plus about 150 keywords. Beware 'C', 'Go' and bare words like 'rest'.
- Parse salary per month or per year to LPA (conversion is an estimate); detect fresher, intern, apprentice and any named batch year.

Check: 15 sample descriptions pass, including the 'C' and 'Go' traps and four salary formats.

Session 7: Dedup + freshness (~2-3h)

- fingerprint = sha256(company | title | location), all normalized; canonical_url also unique.
- 'New' means the fingerprint is not in seen_jobs. Fresh means posted in the last 48 hours, or date unknown but first seen today.
- Upsert into jobs and seen_jobs.

Check: The same job arriving from two sources counts once.

Session 8: Eligibility + match score (~3h)

- Hard filters: whole-word exclude titles, blacklist, required years above 2, a named batch year other than 2027, full-time salary below 3.5 LPA when listed.
- Score 0 to 10 with weights from config: skills 4, role 3, level 2, location 1 (1.0 preferred city, 0.5 elsewhere in India or remote, 0 abroad).
- Return matched skills, missing skills (tracked gaps shown but not scored), the location reason and short reasons.

Check: Senior roles and a 2025-batch-only listing are excluded; a fresher Python role in Pune ranks high.

Session 9: Digest builder (~2-3h)

- Jinja2 HTML and plain-text templates. Two sections: internships and apprenticeships, then full-time. Each has Strong (7+) and Maybe (4 to 6).
- Each item: apply link first, then title, company, location, salary or 'salary not listed', score, matched and missing skills, source. Up to 40 items.
- Header with counts (N new, M already seen), Remote credit, a note naming any failed source. Save the digest in the database.

Check: A day with zero new jobs still renders a short 'nothing new' digest.

Session 10: Delivery: Telegram and email (~3h)

- Notifier protocol with Telegram and email implementations. Channels come from the profile.
- Telegram: BotFather bot and your chat id; split messages over 4096 characters.
- Email: SMTP port 587 with STARTTLS to NOTIFICATION_EMAIL (mailbox B), HTML plus plain text.
- Record delivery per channel. Mark jobs notified after at least one channel succeeds; one failing channel never blocks the other.

Check: The digest arrives on both channels; with one broken, the other still delivers; sending twice repeats nothing.

Session 11: Scheduler (~2-3h)

- daily-digest.yml with cron '30 2 * * *' (02:30 UTC = 08:00 IST) plus workflow_dispatch.
- Secrets: database URL, PROFILE_YAML, Adzuna, Telegram, SMTP, IMAP. Scheduled runs can start late; check GitHub's current rules on inactivity.
- APScheduler is the local alternative for testing.

Check: A manual run sends a digest; the scheduled run fires by itself the next morning.

Session 12: Tests, tuning, reliability (~3h)

- Mocked-network pytest for every source, normalize, dedup, match, digest and notify.
- Handle zero results, rate limits, a failed source (named in the digest) and a failed channel.
- Watch 3 to 5 real mornings. Tune min score, weights, noisy aliases and the blocklist. Note which sources produce jobs you actually apply to.

Check: Suite is green; real digests received on both channels; you would open it each morning.

Milestone 1: the digest arrives at 8 AM on Telegram and email with no action from you.

9. Phase 2: private app, single user (Sessions 13 to 17)

A password-protected app, hosted online, so you can use the agent from any device. Only you can log in.

Session 13: Profile in database + job status (~3h)

- Move the profile into the database: validated on save, last good version kept, history stored.
- Add job_status (saved, applied, not_interested, with timestamps). The agent reads the profile from the database.
- Remove the PROFILE_YAML secret once the database copy works.

Check: A profile edit changes the next run; an invalid edit falls back to the last good version and says so.

Session 14: App + login + read pages (~3h)

- FastAPI + Jinja2 app. Single-user login: argon2 password hash or Supabase Auth limited to your email; secure session cookie; HTTPS only; login rate limit.
- Pages: today's digest and job history with filters (section, score, location, source).

Check: A wrong password is rejected; every page needs a session.

Session 15: Actions + settings (~3h)

- Buttons for saved, applied and not interested; weekly applied count.
- Settings editor for skills, roles, min score, blocklist and channels; source-health page; manual add-a-job form.

Check: A blocklist edit made from your phone changes tomorrow's digest.

Session 16: Deploy (~2-3h)

- Pick a host (Render, Railway, Fly.io or a small VPS) and check its current terms; free tiers may sleep when idle.
- Environment secrets, HTTPS, subdomain such as app.<yourdomain>, a /health endpoint.

Check: The app loads from your phone on mobile data.

Session 17: Hardening (~2-3h)

- Security headers, cookie flags, rate limiting, noindex (meta + robots Disallow, kept out of the sitemap).
- Uptime check and a database backup plan.
- Optional: install-to-home-screen manifest; a Run now button using a token that can only trigger the one workflow.

Check: Security headers test passes; the app is unreachable without login.

Milestone 2: you can review jobs, mark applied and edit your profile from your phone.

10. Phase 3: public project site (Sessions 18 to 21)

A static project page: what it does, how it works, its sources, limits and real build-time stats. It never shows your digest or profile.

Session 18: Site foundation (~3h)

- site/build.py renders Jinja2 templates into dist/. Pages: Home, How it works, Sources, Privacy Policy, Terms, 404.
- Clean slugs such as /how-it-works/. Exactly one H1 per page. Unique title and meta description per page. Canonical tag on every page.
- A real favicon and a custom browser tab title. Breadcrumbs and internal links between pages.

Check: A build test asserts every page has one H1, a unique title, a description and a canonical.

Session 19: Real content + structured data (~3h)

- Write specific copy. Hand-built SVG architecture diagram with alt text; a redacted real Telegram screenshot with alt text.
- One 1200x630 social-share image per page; Open Graph and Twitter tags.
- JSON-LD: SoftwareApplication, WebSite, BreadcrumbList. (Local Business schema does not apply.)
- Privacy Policy and Terms that describe exactly what the project stores and calls. The public site never shows your digest or profile.

Check: Schema passes the schema.org validator and Google's Rich Results Test; grep finds no 'lorem', 'TODO' or placeholder text.

Session 20: Domain + crawl files (~2-3h)

- Generate sitemap.xml and robots.txt at build time; add llms.txt (optional convention, no confirmed ranking benefit).
- Custom 404 page. Register a domain, set DNS for GitHub Pages, enforce HTTPS, add pages.yml to deploy dist/ on push.
- Note: GitHub Pages from a private repo may need a paid plan; check the current rules.

Check: The domain serves over HTTPS; /sitemap.xml, /robots.txt and /llms.txt resolve; a bad URL shows your 404.

Session 21: Audit + release (~3h)

- Run Lighthouse and fix what it flags. Confirm zero console errors in DevTools.
- Confirm no .map files ship in production. Keep JS to a tiny vanilla script or none; record bundle sizes.
- Run the anti-slop checklist. Finish the README: architecture, setup, .env.example explained, limitations. Tag v1.0.

Check: Every checklist item is green and every claim on the site matches the code.

11. Anti-slop rules for the site

Area	Rule
Palette	One accent color on a neutral background. No purple gradients.
Buttons	Normal rounded rectangles, not pills everywhere.
Icons	None, or one consistent set. No emoji as icons.
Motion	No scroll animations, no cursor effects.
Numbers	Only real ones, generated at build time from source_runs and seen_jobs, with a last-updated date. No fake counters, reviews or testimonials.
Hero copy	Concrete, for example: 'Checks company job boards and Adzuna at 08:00 IST, drops duplicates, and sends matches to Telegram and email.' Not 'Supercharge your career journey.'
Images	Your own screenshots and diagrams only. No generated stock photos.
Prose	Write it yourself or edit heavily. Keep dashes to a minimum.
Branding	No 'Made with AI' footer. A 'Source on GitHub' link is enough.

12. Pre-launch audit checklist

Item	Session	Done
Custom domain, HTTPS	20	☐
Clean URL slugs	18	☐
Custom 404 page	20	☐
Unique page titles	18	☐
Meta descriptions	18	☐
Canonical tags	18	☐
One H1 per page	18	☐
sitemap.xml, robots.txt, llms.txt	20	☐
Favicon, custom tab title	18	☐
Internal links, breadcrumbs	18	☐
Structured data (schema)	19	☐
Social-share images	19	☐
Alt text on images	19	☐
Privacy Policy, Terms	19	☐
No placeholder content	19	☐
Zero console errors	21	☐
No source maps in production	21	☐
Small JS bundles	21	☐
Anti-slop rules applied	21	☐
Private app noindex, behind login	17	☐

13. Risks and fallbacks

Risk	Fallback
A source changes or blocks	Sources are isolated; failed sources are named in the digest; swap adapters without touching the rest.
Email alert layouts change	Keep saved sample emails as tests; fix one parser at a time.
Scheduled run is late or paused	Use workflow_dispatch; keep repo activity; APScheduler as a local alternative.
Too many weak matches	Raise min score, trim noisy aliases, add to the blocklist after watching 3 to 5 mornings.
Salary rarely listed	Keep unlisted jobs and label them; filter only when pay is stated.
Alias matches ordinary words	Whole-word matching, tests for traps such as 'C', 'Go' and 'rest'.
Secrets leak	Secrets only in .env and repository secrets; scan history; rotate anything that leaks.
Free host sleeps	Accept a slow first load, or use a cheap always-on host.

14. Verify on the day you build

- Supabase pooler string works from GitHub Actions (Session 1).
- Adzuna, Jooble and Remotive current limits, terms and attribution rules (Sessions 4).
- Workable's public endpoint and terms (Session 3).
- Google OAuth refresh-token expiry for apps in testing mode, if you ever choose the Gmail API over IMAP (Session 5).
- GitHub Actions scheduled-workflow and Pages rules for your repo visibility (Sessions 11, 20).
- Host free-tier limits and sleep behavior (Session 16).

15. Daily checklist

- Read today's card and write the goal in one line before coding.
- Build only today's deliverable. Write the tests first or together with the code.
- Run the Check line. If it fails, fix it before moving on.
- Commit and push. Update README or decisions.md if a decision changed.
- Write down tomorrow's first step so you can start quickly.

16. What comes next

The AI Coding Agent stays a separate project, to be built after the Job Digest has run on its own for a few mornings. Rebuild its plan from the original day cards, restore the benchmark comparison runs, and rename the 'unverified' status so it does not contain the banned word.